εξέλθει από τον βρόχο. Όμως, η continue κατευθύνει άμεσα τη ροή στη συνθήκη ελέγχου η οποία φυσικά για την τιμή του i ίση με το 1 είναι ψευδής και έτσι ο βρόχος διακόπτεται έχοντας εκτυπώσει μόνο το 1.

5.2.3 Η επαναληπτική δομή for

Η for παρουσιάζει μεγάλη ευελιξία και είναι ίσως η συχνότερα χρησιμοποιούμενη επαναληπτική δομή. Η γενική μορφή της έχει ως εξής:

```
for (initialization; condition; statement) {
     statement(s);
}
```

Η for επομένως αποτελείται από τέσσερα μέρη. Το initialization είναι το τμήμα εκείνο στο οποίο τυπικά αρχικοποιείται ή και δηλώνεται η μεταβλητή ελέγχου του βρόχου. Η condition είναι η συνθήκη που ελέγχει αν θα συνεχιστεί η επαναληπτική διαδικασία ή όχι. Το statement μπορεί να είναι μια οποιαδήποτε εντολή ή συνάρτηση. Στην τυπική περίπτωση, στο statement μεταβάλλεται η τιμή της μεταβλητής ελέγχου. Το τέταρτο μέρος είναι το μπλοκ της for στο οποίο μπορούμε να τοποθετήσουμε ένα ή περισσότερα statements.

Το initialization εκτελείται μια φορά κατά την είσοδο στον βρόχο. Στη συνέχεια εκτελείται η condition. Αν βρεθεί αληθής, η ροή κατευθύνεται στο μπλοκ της for. Αφού εκτελεστούν οι statements του μπλοκ, η ροή κατευθύνεται στο statement εντός του κύριου σώματος της for. Μετά την εκτέλεση του statement εντός του κύριου σώματος, η ροή κατευθύνεται στην condition για να ακολουθήσει μια κυκλική διαδρομή του τύπου condition-statement(s)-statement έως ότου βέβαια η condition λάβει την τιμή false. Σε αυτήν την περίπτωση, διακόπτεται η for και η ροή του προγράμματος κατευθύνεται στην έξοδο του βρόχου.
Ας δούμε καταρχάς μια απλή εφαρμογή της for. Ο κώδικας 5.18, τυπώνει τους ακεραίους από 0 έως 9.

```
for (int i = 0; i < 10; i++) {
     System.out.println(i);
}
```

Κώδικας 5.18 *Εκτύπωση των ακέραιων 0..9 με την for*

Η πρώτη τιμή που εκτυπώνεται είναι η τιμή 0. Αυτό δείχνει πως η ροή εκτέλεσης πρώτα εισέρχεται στο μπλοκ της for και στη συνέχεια εκτελεί την προσαύξηση του i στο κυρίως σώμα της for. Η δε τελευταία τιμή είναι το 9. Πράγματι, μόλις τυπωθεί το 9, η ροή κατευθύνεται στην πρόταση προσαύξησης του i που γίνεται 10. Στη συνέχεια, η συνθήκη ελέγχου καθίσταται ψευδής και οδηγεί στην έξοδο του βρόχου.
Ας σημειωθεί πως στο τμήμα αρχικοποίησης της for δεν είναι υποχρεωτικό να δηλώνεται η μεταβλητή ελέγχου. Ωστόσο, αποτελεί καλή πρακτική η δήλωσή της εφόσον αυτή δεν είναι χρήσιμη έξω από την for. Κάτι ακόμη που πρέπει να αποσαφηνίσουμε είναι πως κανένα από τα τέσσερα τμήματα της for δεν είναι υποχρεωτικό.

Στη συνέχεια δίνουμε παραδείγματα χρήσης της for μέσα από τα οποία γίνονται σαφείς οι παρατηρήσεις μας σε σχέση με αυτήν. Πιο συγκεκριμένα, υλοποιούμε με for κώδικες που αξιοποιούν την while ή την do-while και έχουν ήδη παρουσιαστεί μέχρι εδώ σε αυτήν την ενότητα. Με αυτόν τον τρόπο θα αποκτήσουμε πληρέστερη εικόνα της σχετικής ευελιξίας της for.
Ο κώδικας 5.10 που επιδεικνύει τη χρήση της while, υλοποιείται με for όπως δείχνει ο κώδικας 5.19.

```
static void forDemo() { //Κώδικας 5.19
    int i;
    for (i = 10; i > 0; i--) {
        System.out.print("i>0 ");
    }
    System.out.println("i=" + i);
}
```

Κώδικας 5.19 *Ίδιο αποτέλεσμα με την while του κώδικα 5.10*

Στον κώδικα 5.19 η μεταβλητή ελέγχου i δεν δηλώνεται στο τμήμα αρχικοποίησης της for. Αυτό είναι αναγκαίο, καθώς θέλουμε να προσπελάσουμε τη μεταβλητή έξω από τα όρια της for. Αν την είχαμε δηλώσει στο τμήμα αρχικοποίησης της for, η εμβέλεια της μεταβλητής θα περιοριζόταν στα τμήματα της for.

Η συνάρτηση prtRandoms που παρουσιάζεται στον κώδικα 5.12 μεταγράφεται με for όπως φαίνεται στον κώδικα 5.20:

```
static void prtRandoms() { //Κώδικας 5.20
    Random generator = new Random();
    int rI;
    for (int i = 0; i++ < 10;) {
        rI = generator.nextInt(10) + 1;
        System.out.println(rI);
        if (rI < 2) {
            break;
        }
    }
}
```

Κώδικας 5.20 *Η prtRandoms του κώδικα 5.12 υλοποιημένη με for*

Προσέξτε πως σε αυτήν την περίπτωση το τρίτο τμήμα της for δεν χρησιμοποιήθηκε. Όπως αναφέραμε προηγουμένως, όλα τα τμήματα της for είναι προαιρετικά.
Η συνάρτηση prtRandoms που παρουσιάστηκε στον κώδικα 5.13 μεταγράφεται με for όπως δείχνει ο κώδικας 5.21:

```
static void prtRandoms() { //Κώδικας 5.21
    int rI;
    Random generator = new Random();
    for (int i = 0; i < 10; i++) {
        rI = generator.nextInt(10) + 1;
        if (rI == 3 || rI == 4) {
            continue;
        }
        System.out.println(rI);
    }
}
```

Κώδικας 5.21 *Η prtRandoms του κώδικα 5.13 υλοποιημένη με for*

Ο κώδικας 5.22 υλοποιεί την calcSum του κώδικα 5.14 χρησιμοποιώντας for αντί για do-while:

```
static void calcSum() { //Κωδικας 5.22
    int sum = 0;
    Scanner input = new Scanner(System.in);
    for (int i = 1; i != 0;) {
        System.out.print("Enter int (0 terminates input):");
        i = input.nextInt();
        sum += i;
    }
    System.out.println("Sum = " + sum);
}
```

Κώδικας 5.22 *Η calcSum του κώδικα 5.14 υλοποιημένη με for*

Ο κώδικας 5.23 παρέχει μια κάπως διαφορετική υλοποίηση του κώδικα 5.18:

```
 for (int i=0; i<10; System.out.println(i));
```

Κώδικας 5.23 *Εναλλακτική υλοποίηση του κώδικα 5.18*

Συχνά οι αρχάριοι προγραμματιστές έχουν την εντύπωση πως οι μεταβλητές ελέγχου της for είναι υποχρεωτικά τύπου int. Κάτι τέτοιο βέβαια δεν αληθεύει. Παρακάτω στον κώδικα 5.24, εκτυπώνουμε το αγγλικό αλφάβητο με κεφαλαίους χαρακτήρες:

```
static void prtAlpha() { //Κώδικας 5.24
```

```
        for (char c = 'A'; c <= 'Z'; c++) {
            System.out.print(c + " ");
        }
        System.out.println();
    }
```

Κώδικας 5.24 *Εκτύπωση της Αγγλικής Αλφάβητου με for*

Η μεταβλητή που αρχικοποιείται στο τμήμα αρχικοποίησης της for δεν λαμβάνει υποχρεωτικά μέρος στη συνθήκη ελέγχου. Αυτό επιδεικνύεται στον κώδικα 5.25, ο οποίος τυπώνει διαδοχικούς ακέραιους αρχίζοντας από το 0 έως ότου ο χρήστης επιλέξει έξοδο.

```
    static void prtInts() {//Κώδικας 5.25
        int v = 1;
        Scanner input = new Scanner(System.in);
        for (int i = 0; v > 0; i++) {
            System.out.println(i);
            System.out.print("Εισάγετε 0 για έξοδο:");
            v = input.nextInt();
        }
    }
```

Κώδικας 5.25 *Εκτύπωση διαδοχικών ακέραιων μέχρι να επιλέξει ο χρήστης έξοδο*

Όπως στις δομές επιλογής, έτσι και εδώ έχουμε τη δυνατότητα εμφωλευμένων επαναληπτικών δομών. Ο κώδικας 5.26 τυπώνει 10 φορές το αγγλικό αλφάβητο.

```
    static void nestedFor() { //Κώδικας 5.26
        for (int i = 0; i < 10; i++) {
            for (char c = 'A'; c <= 'Z'; c++) {
                System.out.print(c + " ");
            }
            System.out.println();
        }
    }
```

Κώδικας 5.26 *Παράδειγμα εμφωλευμένης for*

Ο κώδικας 5.27 έχει ως σκοπό να δώσει έμφαση στο γεγονός ότι τα διάφορα τμήματα της for είναι προαιρετικά. Τέτοιου είδους χρήση της for υποστηρίζεται μεν από τον μεταγλωττιστή, είναι ωστόσο ασυνήθιστη.

```
    static void emptyFor() { //Κώδικας 5.27
        int i = 0;
        for (;;) {
            System.out.println(i++);
            if (i > 10) {
                break;
            }
        }
    }
```

Κώδικας 5.27 *Παράδειγμα της for χωρίς κανένα από τα τρία κύρια τμήματα*

Σημείωση: Υπάρχει και μια επιπλέον μορφή της for, γνωστή ως ενισχυμένη (enhanced) for. Η ενισχυμένη for σχετίζεται με τις συλλογές, δηλαδή με δομές που είναι κατάλληλες για αποθήκευση πλήθους δεδομένων, π.χ. πίνακες. Η ενισχυμένη for παρουσιάζεται στην ενότητα 6.1.2.

5.3. Λυμένες Ασκήσεις

5.3.1 Εκτύπωση πραγματικών με δεκαδικό βήμα

Αναπτύξτε συνάρτηση που τυπώνει τους πραγματικούς αριθμούς από 1 έως 10 με βήμα 0.1.

Λύση
Μια απλή for με μεταβλητή ελέγχου τύπου double αρκεί για τη λύση αυτής της άσκησης.

```
static void simpleForwithDouble() { //Κώδικας 5.28
    for (double i = 0; i <= 10; i += 0.1) {
        System.out.println(i);
    }
}
```

Κώδικας 5.28 *Απλή for με μεταβλητή ελέγχου double και βήμα 0.1*

5.3.2 Εκτύπωση ακέραιων υπό συνθήκη

Να γράψετε πρόγραμμα που τυπώνει όλους τους ακέραιους από το 0..100 που διαιρούνται ακριβώς με το 4, το 6 και το 8.

Λύση
Η πρώτη σκέψη που μας έρχεται στο μυαλό είναι να ελέγξουμε κάθε ακέραιο από το 0 έως το 100 και αν διαιρείται ακριβώς με το 4, το 6 και το 8 να το εμφανίσουμε. Ωστόσο, υπάρχει και πιο αποδοτικός τρόπος. Αφού οι αριθμοί που θα τυπώνουμε διαιρούνται με το 4, το 6 και το 8, μπορούμε να ξεκινήσουμε από το 4 και να ελέγξουμε μόνο τα πολλαπλάσια του 4. Ελέγχουμε λοιπόν τα πολλαπλάσια του 4. Αν πληρούν και τις λοιπές προϋποθέσεις, τότε τα τυπώνουμε. Αν θεωρήσουμε πως και το 0 είναι πολλαπλάσιο του 4, τότε θα ξεκινήσουμε από το 0. Με αυτόν τον τρόπο, αντί να ελέγξουμε 100 περίπου ακέραιους, ελέγχουμε μόνο 25. Μπορούμε και εδώ να χρησιμοποιήσουμε μια for.

```
static void prtRealsConditional() { //Κώδικας 5.29
    for (int i = 0; i <= 100; i += 4) {
        if (i % 6 == 0 && i % 8 == 0) {
            System.out.println(i);
        }
    }
}
```

Κώδικας 5.29 *Εκτύπωση ακέραιων υπό συνθήκη*

5.3.3 Εκτύπωση και καταμέτρηση ακέραιων υπό συνθήκη

Να γράψετε πρόγραμμα που τυπώνει όσους ακεραίους στην περιοχή 0..1000 είναι πολλαπλάσια του 3 και ταυτοχρόνως τέλεια τετράγωνα, δηλαδή η τετραγωνική τους ρίζα είναι ακέραιος.
Μετράει και εκτυπώνει το πλήθος των ακέραιων που πληρούν τις περιγραφόμενες προϋποθέσεις.
Σημείωση: Η τετραγωνική ρίζα μιας αριθμητικής μεταβλητής i δίνεται από τη συνάρτηση sqrt της κλάσης Math : Math.sqrt(i).

Λύση
Το ερώτημα εδώ είναι ο έλεγχος για να διαπιστώσουμε αν η τετραγωνική ρίζα ενός αριθμού είναι ακέραιος. Καταρχάς την τετραγωνική ρίζα θα την υπολογίσουμε με τη συνάρτηση sqrt() της Math. Όπως είναι λογικό, η συνάρτηση αυτή επιστρέφει αριθμό τύπου double. Για να διαπιστώσουμε αν ο αριθμός αυτός είναι ακέραιος, θα ελέγξουμε αν το υπόλοιπο της διαίρεσης μεταξύ τετραγωνικής ρίζας και 1 είναι ίσο με το 0. Προκειμένου όμως να ελέγξουμε σωστά θα πρέπει να χρησιμοποιήσουμε προσεγγιστική ισότητα, όπως έχουμε επισημάνει στις ενότητες 3.2 και 4.1.4.1.

```
static boolean approximateEquals(double f1, double f2, double epsilon) {
    return Math.abs(f2 - f1) < epsilon;
}
```

```
    static void search() { //Κώδικας 5.30
        int cnt = 0;
        for (int i = 0; i < 1000; i += 3) {
            if (approximateEquals(Math.sqrt(i) % 1, 0, 0.00001d)) {
                System.out.print(i + " ");
                cnt++;
            }
        }
        System.out.println();
        System.out.println("Ta teleia tetragwna twn pollaplasiwn tou 3 apo
0..1000 einai " + cnt);
    }
```

Κώδικας 5.30 *Αναζήτηση σε πίνακα πραγματικών με προσεγγιστική ισότητα*

5.3.4 Τυχαίοι ακέραιοι και switch

Να γράψετε πρόγραμμα που λαμβάνει 2 ακέραιους από τον χρήστη και ανάλογα με τις τιμές τους, τυπώνει ένα μήνυμα της μορφής "εποχή ημέρα". Η ονομασία ημέρας και εποχής με βάση τις ακόλουθες αντιστοιχίσεις: (1->Κυριακή, 2->Δευτέρα, …, 7 Σάββατο), (1-> Άνοιξη, 2-> Καλοκαίρι, 3-> Φθινόπωρο, 4->Χειμώνας). Σε περίπτωση που η τιμή ενός (ή και των δύο) ακέραιων είναι εκτός των προαναφερόμενων περιοχών τυπώνει unkown value.

Λύση
Εδώ είναι βολικό να υπολογίσουμε πρώτα το μήνυμα και μετά να το τυπώσουμε. Για τον σκοπό αυτό θα χρησιμοποιήσουμε κατάλληλες μεταβλητές τύπου String.

```
    static void decodeDaySeason() { //Κώδικας 5.31
        Scanner in = new Scanner(System.in);
        System.out.print("Εισάγετε ΑΑ ημέρας: ");
        int day = in.nextInt();
        System.out.print("Εισάγετε ΑΑ εποχής: ");
        int season = in.nextInt();
        String message = "";
        if (day < 1 || day > 7 || season < 1 || season > 4) {
            message = "Unkown Value";
        } else {
            switch (day) {
                case 1:
                    message = "Sunday";
                    break;
                case 2:
                    message = "Monday";
                    break;
                case 3:
                    message = "Tuesday";
                    break;
                case 4:
                    message = "Wensday";
                    break;
                case 5:
                    message = "Thursday";
                    break;
                case 6:
                    message = "Friday";
                    break;
                case 7:
                    message = "Saturday";
                    break;
```

```
            }
            switch (season) {
                case 1:
                    message += " Spring";
                    break;
                case 2:
                    message += " Summer";
                    break;
                case 3:
                    message += " Autumn";
                    break;
                case 4:
                    message += " Winter";
                    break;
            }
        }
        System.out.println(message);
    }
```

Κώδικας 5.31 *Εκτύπωση ημέρας και εποχής*

Στον κώδικα 5.31, καταρχάς, εισάγουμε από το πληκτρολόγιο δύο ακέραιους, ο ένας αντιπροσωπεύει τον αύξοντα αριθμό της ημέρας και ο άλλος της εποχής. Στη συνέχεια δηλώνουμε και αρχικοποιούμε τη μεταβλητή message τύπου String. Η αρχικοποίηση στη συγκεκριμένη περίπτωση είναι υποχρεωτική. Αν δεν αρχικοποιήσουμε την message, ο μεταγλωττιστής θα παράγει σφάλμα σε όλα τα σημεία του κώδικα που επιχειρείται η σύνδεση της message με την περιγραφή της εποχής. Πριν μπούμε στον έλεγχο της ημέρας στην πρώτη switch, ελέγχουμε αν οι αύξοντες αριθμοί είναι στα επιτρεπτά όρια. Αν κάποιος από τους ακέραιους που θα εισάγει ο χρήστης είναι εκτός ορίων, τότε το message έχει λάβει τιμή. Επιπλέον, η ροή του κώδικα δεν θα περάσει από τις επίμαχες γραμμές. Αν πάλι κανένας από τους ακέραιους δεν είναι εκτός ορίων, το message θα έχει λάβει τιμή από την πρώτη switch. Επομένως, το message θα έχει λάβει οπωσδήποτε τιμή πριν η ροή κατευθυνθεί στη δεύτερη switch. Ωστόσο, το αποτέλεσμα δεν μπορεί να διαπιστωθεί κατά τον χρόνο μεταγλώττισης. Για τον μεταγλωττιστή είναι πιθανό η ροή να μπει στη δεύτερη switch χωρίς να έχει αρχικοποιηθεί το message. Για παράδειγμα, αν δεν ελέγξουμε την day για την τιμή 1, αλλά την ελέγξουμε για μια άλλη τιμή, π.χ. 9, τότε το message μπορεί να φτάσει στη δεύτερη switch αναρχικοποίητο.

5.3.5 Έλεγχος αν ο ακέραιος είναι πρώτος ή όχι

Να γράψετε πρόγραμμα που λαμβάνει έναν ακέραιο από τον χρήστη και εξετάζει αν είναι πρώτος αριθμός ή όχι.
Σημείωση: Πρώτος είναι ένας ακέραιος μεγαλύτερος της μονάδας που διαιρείται ακριβώς μόνο με το 1 και τον εαυτό του.

Λύση
Όλοι οι ακέραιοι, πρώτοι και μη, διαιρούνται ακριβώς με το 1 και τον εαυτό τους. Αν όμως ένας ακέραιος διαιρείται ακριβώς με οποιονδήποτε άλλον πλην του 1 και του εαυτού του, τότε δεν είναι πρώτος. Επομένως θα πρέπει να εξετάσουμε αν ο ακέραιος που θα εισάγει ο χρήστης διαιρείται ακριβώς με οποιονδήποτε άλλον ακέραιο. Αν θεωρήσουμε έναν ακέραιο x, ποιοι είναι οι ακέραιοι εκτός του 1 και του x που έχουν πιθανότητα να διαιρούν ακριβώς τον x; Είναι προφανές πως οι μεγαλύτεροι του x δεν έχουν τέτοια πιθανότητα. Επίσης αποκλείονται το 1 και το x. Επομένως μένουν οι ακέραιοι από το 2 έως x-1. Πράγματι αν εξετάσουμε αν το x διαιρείται ακριβώς με οποιονδήποτε ακέραιο από 2 έως x-1, μπορούμε να διαπιστώσουμε αν ο x είναι πρώτος ή όχι. Αν όμως σκεφτούμε λίγο πιο προσεκτικά, θα αντιληφθούμε πως οι αριθμοί που είναι μεγαλύτεροι από x/2 επίσης αποκλείεται να διαιρούν ακριβώς τον x καθώς ο μεγαλύτερος πιθανός διαιρέτης του είναι ο x/2. Συνεπώς αρκεί να εξετάσουμε τους ακέραιους από 2 έως και x/2.

```
    static void prwtos() { //Κώδικας 5.32
        Scanner in = new Scanner(System.in);
        System.out.print("Εισάγετε ακέραιο: ");
        int k = in.nextInt();
```

```
        boolean isPrwtos = k >= 2;
        if (isPrwtos) {
            for (int i = 2; i <= k / 2; i++) {
                if (k % i == 0) {
                    isPrwtos = false;
                    break;
                }
            }
        }
        if (isPrwtos) {
            System.out.println(k + " είναι πρώτος");
        } else {
            System.out.println(k + " δεν είναι πρώτος");
        }
    }
```

Κώδικας 5.32 *Έλεγχος αν ένας ακέραιος είναι πρώτος αριθμός*

Καταρχάς λαμβάνουμε την είσοδο του χρήστη στην ακέραια μεταβλητή k. Στη συνέχεια, δηλώνουμε και αρχικοποιούμε τη μεταβλητή isPrwtos τύπου boolean. Η isPrwtos αρχικοποιείται στην τιμή της έκφρασης k>=2, δηλαδή στην τιμή true αν το k είναι μεγαλύτερο ή ίσο του 2 και στην τιμή false αν το k είναι μικρότερο από το 2. Στη δεύτερη αυτή περίπτωση, το k εξ ορισμού δεν είναι πρώτος αριθμός. Επομένως ο κώδικας δεν θα μπει καθόλου στην if που ακολουθεί αλλά θα μεταβεί κατευθείαν στην else της επόμενης if και θα τυπώσει πως ο αριθμός δεν είναι πρώτος.

Αντίθετα, αν το k είναι μεγαλύτερο ή ίσο με το 2, η ροή του κώδικα θα εισέλθει στην πρώτη if. Στη συνέχεια ο έλεγχος περνάει στην επαναληπτική διαδικασία που ακολουθεί. Ο σκοπός αυτού του βρόχου είναι να ελέγξει αν το k διαιρείται ακριβώς με οποιονδήποτε ακέραιο από το 2 έως το k/2. Αν βρεθεί έστω ένας ακέραιος που διαιρεί ακριβώς το k, τότε το k δεν είναι πρώτος οπότε η isPrwtos ενημερώνεται με την τιμή false και η επαναληπτική διαδικασία διακόπτεται. Στη συνέχεια η ροή κατευθύνεται στη δεύτερη if η οποία ανάλογα με την τιμή της isPrwtos τυπώνει το κατάλληλο μήνυμα.

5.3.6 Εκτύπωση Αγγλικής αλφάβητου

Να γράψετε πρόγραμμα που τυπώνει 100 φορές την Αγγλική αλφάβητο με πεζούς χαρακτήρες αντεστραμμένη, δηλαδή από το z στο a. Δώστε 2 λύσεις. Η πρώτη να χρησιμοποιεί μόνο for και η δεύτερη να χρησιμοποιεί συνδυασμό while και do-while.

Λύση
Ακολουθούν οι ζητούμενες υλοποιήσεις:

```
    static void printReversedAlphabet() { //Κώδικας 5.33
        for (int I = 0; i < 100; i++) {
            for (char c ='{'; c >='a'; c--) {
                System.out.print(c +"""");
            }
            System.out.println();
        }
    }

    static void printReversedAlphabet2() { //Κώδικας 5.33
        int i = 0;
        while (i++ < 100) {
            char c ='{';
            do {
                System.out.print(c +"""");
                c--;
            } while (c >='a');
            System.out.println();
```

```
        }
    }
```

Κώδικας 5.33 *Εκτύπωσητιςς Αγγλικής αλφαβήτου με αντεστραμμένη διάταξη. Δύο προσεγγίσεις.*

5.3.7 Εκθετική εξίσωση

Γράψτε κατάλληλο κώδικα που υπολογίζει τις τιμές των a,b,c στην $2^a*3^b*5^c=22500$, με a<20, b<20 και c<20. Σε περίπτωση που δεν βρεθεί λύση, το πρόγραμμα να εμφανίζει το μήνυμα "Δεν βρέθηκε λύση".

Λύση
Ο πιο άμεσος τρόπος για τη λύση αυτής της άσκησης είναι η χρήση εμφωλευμένων for βάθους τριών επιπέδων. Θα πρέπει όμως να προσέξουμε ώστε να χρησιμοποιήσουμε έλεγχο προσεγγιστικής ισότητας για τη σύγκριση πραγματικών αριθμών.

```
    static void ekthetikiExiswsi() { //Κώδικας 5.34
        boolean solutionFound = false;

        for (int a = 0; a < 20; a++) {
            for (int b = 0; b < 20; b++) {
                for (int c = 0; c < 20; c++) {
                    double rslt = Math.pow(2, a) * Math.pow(3, b) * Math.pow(5,
c);
                    if (approximateEquals(rslt, 22500, 0.000001)) {
                        solutionFound = true;
                        System.out.println("a=" + a + ", b=" + b + ", c=" + c);
                        break;
                    }
                }
            }
        }
        if (!solutionFound) {
            System.out.println("Δεν βρέθηκε λύση");
        }
    }
```

Κώδικας 5.34 *Λύση εξίσωσης*

Μία σημαντική παρατήρηση εδώ είναι πως η break μεταφέρει τη ροή του προγράμματος έξω από τον εξωτερικό βρόχο και όχι απλά έξω από τον εσωτερικό βρόχο.

5.4 Ασκήσεις προς λύση

1. Αναπτύξτε πρόγραμμα που δέχεται έναν ακέραιο ως είσοδο από τον χρήστη και τυπώνει όλους τους διαιρέτες του.

2. Αναπτύξτε πρόγραμμα που λαμβάνει δύο ακεραίους από τον χρήστη και τυπώνει τον Μέγιστο Κοινό Διαιρέτη τους.

3. Αναπτύξτε πρόγραμμα που λαμβάνει έναν ακέραιο x από τον χρήστη και τυπώνει όλους τους πρώτους αριθμούς που περιλαμβάνονται μεταξύ του 2 και του x.

4. Αναπτύξτε πρόγραμμα που παράγει ερωτήσεις πρόσθεσης ή αφαίρεσης με τυχαίους τελεστέους, ζητά τις απαντήσεις από τον χρήστη και παρέχει αξιολόγηση των απαντήσεων στη μορφή:

```
Απάντησες σωστά σε k από m ερωτήσεις πρόσθεσης
Απάντησες σωστά σε r από t ερωτήσεις αφαίρεσης
Μέσος όρος βαθμολογίας : y
```

5. Αναπτύξτε πρόγραμμα που ζητά από τον χρήστη των αριθμό ωρών που εργάστηκε κάθε ημέρα μιας εβδομάδας, το ωρομίσθιό του και υπολογίζει:

1. Ποια ημέρα εργάστηκε τις περισσότερες ώρες;
2. Ποια ημέρα εργάστηκε τις λιγότερες ώρες;
3. Πόσες ώρες κατά μέσο όρο εργάστηκε την κάθε ημέρα της εβδομάδας;
4. Ποια είναι η αμοιβή της εβδομάδας σύμφωνα με τις ώρες που εργάστηκε και το ωρομίσθιό του;

6. Αναπτύξτε πρόγραμμα που λαμβάνει ένα αριθμό από τον χρήστη, έστω n, και υπολογίζει τον μικρότερο ακέραιο k για τον οποίο ισχύει $k^2>n$

7. Αναπτύξτε πρόγραμμα που λαμβάνει είσοδο από τον χρήστη μια ακέραια τιμή που αναπαριστά ένα έτος και υπολογίζει αν το έτος είναι δίσεκτο. Βρείτε τον ορισμό του δίσεκτου έτους στο διαδίκτυο.

8. Ένας πωλητής λαμβάνει μηνιαίο μισθό 1200 Ευρώ. Επιπλέον, του μισθού λαμβάνει προμήθεια επί των πωλήσεων. Για πωλήσεις από 0 έως 10.000 Ευρώ, η προμήθεια είναι 5%. Για πωλήσεις από 10.000 έως 20.000 Ευρώ, η προμήθεια είναι 10%. Για πωλήσεις από 20.000 Ευρώ και πάνω, η προμήθεια είναι 20%. Επομένως, ο πωλητής για έναν μήνα που πραγματοποίησε πωλήσεις 15.000 Ευρώ θα λάβει προμήθεια ίση με 10.000x5%+5.000x10%. Αναπτύξτε πρόγραμμα που ζητά από τον πωλητή τις πωλήσεις για κάθε μήνα ενός έτους και υπολογίζει και εκτυπώνει τόσο την μηνιαία αμοιβή του όσο και την ετήσια.

9. Η εταιρία Service απασχολεί τρεις κατηγορίες εργαζόμενων, κατόχους απολυτηρίου Λυκείου, κατόχους Πανεπιστημιακού πτυχίου και κατόχους Διδακτορικού τίτλου. Τα ωρομίσθια των εργαζόμενων έχουν ως εξής: 40 Ευρώ για τους κατόχους απολυτηρίου Λυκείου, 60 Ευρώ για τους κατόχους πανεπιστημιακού πτυχίου και 80 Ευρώ για τους κατόχους Διδακτορικού τίτλου. Το τυπικό ωράριο για κάθε εργαζόμενο είναι 40 ώρες την εβδομάδα. Κάθε επιπλέον ώρα εργασίας αμείβεται με προσαύξηση 50% επί του ωρομισθίου του εργαζόμενου. Γράψτε πρόγραμμα που ζητά από τον εργαζόμενο την κατηγορία στην οποία ανήκει, τον αριθμό ωρών που εργάστηκε την προηγούμενη εβδομάδα και υπολογίζει πόσες είναι οι τακτικές αποδοχές της εβδομάδας, πόσες οι αποδοχές λόγω υπερωρίας και ποιο το σύνολο των αποδοχών.

10. Αναπτύξτε συνάρτηση που λαμβάνει έναν ακέραιο από τον χρήστη τον οποίο εκχωρεί σε κατάλληλη μεταβλητή αφού διασφαλίσει πως η εισαγόμενη τιμή βρίσκεται στα όρια 1 έως 10.

11. Μελετήστε τις συναρτήσεις

```
static void chkBoolean() {
     Random r = new Random();
     boolean b = r.nextBoolean();
     if (b == true) {
          System.out.println("Επιτυχία!");
     } else {
          System.out.println("Αποτυχία");
     }
}

static void chkBoolean2() {
     Random r = new Random();
     boolean b = r.nextBoolean();
     if (b) {
          System.out.println("Επιτυχία!");
     } else {
          System.out.println("Αποτυχία");
     }
}
```

Έχουν κάποια διαφορά ως προς το αποτέλεσμα; Ποια θα προτιμούσατε και γιατί;

12. Μελετήστε τον ακόλουθο κώδικα

```
static void bugAlphabet() { Άσκηση 5.20
     for (char c = 'z'; c >= 'a'; c--) {
          System.out.print(c + ' ');
     }
}
```

Ποια είναι η έξοδός του; Τρέξτε τον, να διαπιστώσετε αν προβλέψατε τη σωστή έξοδο. Αν όχι, εξηγήστε ποια η διαφορά και γιατί;

14. Να μεταγραφεί η συνάρτηση

```
public static void xEx1() {
     for (int i = 0, j = 1; i < 10 || i < j; i++) {
          j += i;
          if (i % 3 == 0) {
               j = 0;
          }
          System.out.print("(" + i + ", " + j + ") ");
     }
     System.out.println();
}
```

Με αντικατάσταση του for με while

Με αντικατάσταση του for με do while

15. Να μεταγραφεί η συνάρτηση

```
public static void xEx2() {
     int k = 3;
     if (k == 3) {
          System.out.print(3 + " ");
     }
     if (k == 4 || k == 3) {
          System.out.print(4 + " ");
     } else if (k == 5) {
          System.out.print(5 + " ");
     } else {
          System.out.print("unkown");
     }
     System.out.println();
}
```

με χρήση της switch.

Κεφάλαιο 6

Σύνοψη
Στο κεφάλαιο αυτό παρουσιάζεται μια εισαγωγή στους πίνακες. Πιο συγκεκριμένα, περιλαμβάνονται η δημιουργία μονοδιάστατων και πολυδιάστατων πινάκων, η ενισχυμένη for, η δημιουργία αντιγράφων πινάκων, η ταξινόμηση, η σειριακή και δυαδική αναζήτηση, ο έλεγχος ισότητας, η εμφάνιση πίνακα, ο υπολογισμός αθροίσματος των στοιχείων ενός πίνακα, η εύρεση του μεγαλύτερου και του μικρότερου στοιχείου, η αρχικοποίηση με τυχαίες τιμές. Το κεφάλαιο περιέχει λυμένες ασκήσεις και ασκήσεις προς λύση.

Προαπαιτούμενη γνώση
Οι θεμελιώδεις τύποι και η βασική γνώση του τύπου String. Βασική είσοδος και έξοδος. Η λειτουργία των τελεστών. Οι δομές επιλογής και οι επαναληπτικές δομές.

Λέξεις κλειδιά
Μονοδιάστατοι πίνακες, πολυδιάστατοι πίνακες, δείκτης, ταξινόμηση, σειριακή αναζήτηση, δυαδική αναζήτηση.

6 Πίνακες

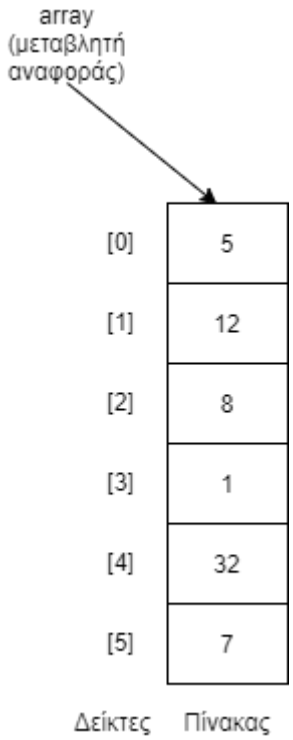
Οι πίνακες είναι συλλογές δεδομένων ίδιου τύπου. Για παράδειγμα, ένας πίνακας ακέραιων αποθηκεύει μια σειρά από τιμές τύπου int ή ένας πίνακας πραγματικών αποθηκεύει μια σειρά από τιμές τύπου double. Όπως εξηγούμε αναλυτικά παρακάτω σε αυτήν την ενότητα, η προσπέλαση στα στοιχεία ενός πίνακα είναι η ταχύτερη δυνατή σε σχέση με την προσπέλαση στα στοιχεία συλλογής οποιουδήποτε άλλου τύπου.

Στο πλαίσιο της Java, ένας πίνακας τα στοιχεία του οποίου δεν είναι πίνακες ονομάζεται μονοδιάστατος πίνακας. Ένας πίνακας τα στοιχεία του οποίου είναι μονοδιάστατοι πίνακες ονομάζεται δισδιάστατος πίνακας. Αντίστοιχα, ένας πίνακας τα στοιχεία του οποίου είναι δισδιάστατοι πίνακες ονομάζεται τρισδιάστατος πίνακας, κόκ. Συχνά οι μη μονοδιάστατοι πίνακες αναφέρονται ως πολυδιάστατοι.

Σε αυτό το κεφάλαιο, παρουσιάζουμε τις συντακτικές λεπτομέρειες ορισμού των μονοδιάστατων και πολυδιάστατων πινάκων, εξηγούμε βασικά στοιχεία της ενσωματωμένης στην Java, κλάσης Arrays που παρέχει μια σειρά από συναρτήσεις για τη διαχείριση πινάκων και παρουσιάζουμε μια σειρά από βασικές επεξεργασίες που αφορούν τους πίνακες.

6.1 Μονοδιάστατοι Πίνακες

Για τη διαχείριση ενός πίνακα, χρειάζεται στο πρόγραμμά μας να έχουμε τουλάχιστον μια μεταβλητή μέσω της οποίας μπορούμε να προσπελάσουμε τον πίνακα και τα στοιχεία του. Στην Java, οι μεταβλητές μέσα από τις οποίες προσπελαύνουμε πίνακες είναι μεταβλητές αναφοράς και όχι τιμής, δηλαδή αποθηκεύουν τη διεύθυνση μνήμης στην οποία είναι κατανεμημένος ο πίνακας. Επιπλέον, τα στοιχεία ενός μονοδιάστατου πίνακα κατανέμονται σε διαδοχικές θέσεις στη μνήμη. Σε αυτό το χαρακτηριστικό οφείλεται η ταχύτατη προσπέλαση των στοιχείων ενός πίνακα.
Στο σχήμα 6.1 παρουσιάζουμε έναν πίνακα ακέραιων.



Σχήμα 6.1 *Αναπαράσταση μονοδιάστατου πίνακα ακέραιων*

Ο πίνακας του σχήματος 6.1 έχει μήκος 6 και περιέχει τους ακέραιους: 5, 12, 8, 1, 32, 7. Η μεταβλητή αναφοράς array περιέχει τη διεύθυνση της αρχής του πίνακα. Οι θέσεις του πίνακα είναι αριθμημένες. Στη θέση 0, για παράδειγμα, είναι τοποθετημένη η τιμή 5, στη θέση 1 είναι τοποθετημένη η τιμή 12, κοκ.

Για να προσπελάσουμε ένα στοιχείο του πίνακα χρειαζόμαστε τρία στοιχεία. Τη μεταβλητή αναφοράς που δείχνει στην αρχή του πίνακα, έναν δείκτη (index) που δείχνει τη θέση μέσα στον πίνακα του στοιχείου που θέλουμε να προσπελάσουμε και τον τελεστή θέσης (index operator) που συμβολίζεται ως []. Σύμφωνα με τα παραπάνω, η έκφραση array[2] προσπελαύνει τη θέση με δείκτη 2 του πίνακα array όπου είναι αποθηκευμένη η ακέραια τιμή 8. Αντίστοιχα, array[3]==1, array[4]==32, κλπ.

Οι δείκτες των πινάκων στην Java αριθμούνται πάντα αρχίζοντας από το μηδέν (zero based). Έτσι, ένας πίνακας με μήκος ν έχει δείκτες από 0 έως ν-1. Ο πίνακας του σχήματος 6.1 για παράδειγμα, έχει μήκος 6 και δείκτες από 0 έως 5. Επομένως, αν επιχειρήσουμε να προσπελάσουμε το στοιχείο array[6] επιχειρούμε στην ουσία να προσπελάσουμε περιοχή έξω από τη μνήμη που έχει δεσμευτεί για τον πίνακα. Τέτοια ενέργεια συνιστά λάθος χρόνου εκτέλεσης και παράγει κατάλληλη εξαίρεση.

Όπως αναφέραμε στην αρχή αυτής της ενότητας, οι πίνακες είναι οι ταχύτερες συλλογές. Πιο συγκεκριμένα, οι πίνακες είναι δομές τυχαίας προσπέλασης (random access), δηλαδή δομές στις οποίες ο χρόνος που απαιτείται για την προσπέλαση οποιουδήποτε στοιχείου είναι σταθερός. Με άλλα λόγια, χρειαζόμαστε τον ίδιο χρόνο για να προσπελάσουμε ένα στοιχείο στη θέση 1000 ενός πίνακα με τον χρόνο που απαιτεί η προσπέλαση στη θέση 0 του πίνακα.

Στον πίνακα του σχήματος 6.1, το στοιχείο στη θέση 0 βρίσκεται στη διεύθυνση μνήμης array+0*Integer.SIZE και το στοιχείο στη θέση 1 βρίσκεται στη διεύθυνση μνήμης array+1*Integer.SIZE. Γενικότερα, το στοιχείο στη θέση k ενός μονοδιάστατου πίνακα ακέραιων βρίσκεται στη διεύθυνση αρχής του πίνακα + k*Integer.SIZE. Επομένως, για να υπολογιστεί η διεύθυνση ενός οποιουδήποτε στοιχείου του πίνακα, χρειαζόμαστε να υπολογίσουμε μόνο μια απλή έκφραση.

Αυτή η ευκολία είναι αποτέλεσμα της διαδοχικής κατανομής στη μνήμη των στοιχείων των μονοδιάστατων πινάκων. Το μειονέκτημα βέβαια της διαδοχικής κατανομής είναι πως μετά τη δημιουργία δεν είναι δυνατόν να μεταβληθεί το μέγεθος του πίνακα. Ωστόσο, αυτό το μειονέκτημα, εύκολα μπορούμε να το παρακάμψουμε όπως θα δούμε παρακάτω στην άσκηση 6.5.7.

6.1.1 Δημιουργία μονοδιάστατων πινάκων